Improving the Spatial Resolution of SDO/HMI Transverse and Line-of-Sight Magnetograms Using GST/NIRIS Data with Machine Learning

Chunhui Xu et al. New Jersey Institute of Technology

1. Introduction

We propose DeepHMI , a machine -learning method for solar image super -resolution, to enhance the transverse and line -of-sight magnetograms of solar active regions (ARs) collected by SDO/HMI to better capture the fine -scale magnetic structures with the ground truth data of GST/NIRIS . In this document, we describe the usage of the trained DeepHMI model to super-resolve the test data. Since model training requires GPUs, it is omitted here.

2. Prediction Workflow

2.1 Load Input Data

Test data are stored as FITS files in the directory data/image/test/. Each file is loaded and sorted by filename.

In [1]:
```
# ========================================================================
#   (c) Copyright 2025
#   All rights reserved
#   Programs written by Chunhui Xu
#   Department of Computer Science
#   New Jersey Institute of Technology
#   University Heights, Newark, NJ 07102, USA
#
#   Permission to use, copy, modify, and distribute this
#   software and its documentation for any purpose and without
#   fee is hereby granted, provided that this copyright
#   notice appears in all copies. Programmer(s) makes no
#   representations about the suitability of this
#   software for any purpose.  It is provided "as is" without
#   express or implied warranty.
# ========================================================================

import os
import torch
import torch.nn as nn
import numpy as np
import einops
from astropy.io import fits
from model import Unet_block
```

In [ ]:
```
torch.set_default_dtype(torch.float32)

num_of_gpus = torch.cuda.device_count()

if torch.cuda.is_available():
        device = torch.device("cuda")
else:
        device = torch.device("cpu")

test_path = "dataset_test/"
model_path = "model_out/"
output_path = "sr_out/"

beta = np.linspace(0.001, 0.02, 1000)
alpha = 1 - beta
alpha_prod = np.cumprod(alpha)
alpha_prod = np.array(alpha_prod, dtype=np.float32)
# print(alpha_prod[-1], beta.shape[0], flush=True)

input_x_test_row = []
input_y_test_row = []
for name in os.listdir(test_path):
    fits_file = fits.open(test_path + name)
    fits_data = fits_file[0].data
    x_data = np.concatenate((np.reshape(fits_data[1], (1, 720, 720)),
                             np.reshape(fits_data[3], (1, 720, 720)),
                             np.reshape(fits_data[5], (1, 720, 720))), 0)
```

```
    y_data = np.concatenate((np.reshape(fits_data[0], (1, 720, 720)),
                             np.reshape(fits_data[2], (1, 720, 720)),
                             np.reshape(fits_data[4], (1, 720, 720))), 0)
    input_x_test_row.append((name, x_data))
    input_y_test_row.append((name, y_data))

input_x_test_row = sorted(input_x_test_row,key = lambda x:x[0])
input_y_test_row = sorted(input_y_test_row,key = lambda x:x[0])

x_hmi_test_data = []
name_list = []
for i in range(len(input_x_test_row)):
  # x_hmi_test_data.append(np.clip(input_x_test_row[i][1][0, :, :], 0, 3000)/1500 - 1)
  name_list.append(input_x_test_row[i][0])
  x_1 = input_x_test_row[i][1][0, :, :]
  x_3 = np.clip(input_x_test_row[i][1][2, :, :], 0, 180)*np.pi/180
  x_z = x_1 * np.cos(x_3)
  x_t = x_1 * np.sin(x_3)
  x_hmi_test_data.append(np.concatenate((np.reshape(x_z/1500, (1, 720, 720)),
                      np.reshape(x_t/750-1, (1, 720, 720))), axis = 0))
x_hmi_test_data = np.array(x_hmi_test_data, dtype=np.float32)

y_gst_test_data = []
for i in range(len(input_y_test_row)):
  # y_gst_test_data.append(np.clip(input_y_test_row[i][1][0, :, :], 0, 5000)/2500 - 1)
  y_1 = input_y_test_row[i][1][0, :, :]
  y_3 = np.pi - np.clip(input_y_test_row[i][1][2, :, :], 0, np.pi)
  y_z = y_1 * np.cos(y_3)
  y_t = y_1 * np.sin(y_3)
  y_gst_test_data.append(np.concatenate((np.reshape(y_z/1500, (1, 720, 720)),
                      np.reshape(y_t/750-1, (1, 720, 720))), axis = 0))
y_gst_test_data = np.array(y_gst_test_data, dtype=np.float32)

x_hmi_test = x_hmi_test_data
y_gst_test = y_gst_test_data

print("data loaded, total sample:", len(name_list))
```

```
data loaded, total sample: 1
```

2.2 Load Model

The DeepHMI model has been saved as .pt file by PyTorch.

In [3]:
```
print("start bz prediction")

model = Unet_block(def_ker=32, res_number=[4, 4, 4, 4, 4, 2], att_sca = 3)
model = nn.DataParallel(model, device_ids=[0])
model.to(device)

try:
    checkpoint = torch.load(model_path + "model_bz")
    model.load_state_dict(checkpoint)
except:
    checkpoint = torch.load(model_path + "model_bz", map_location=torch.device('cpu'))
    model.load_state_dict(checkpoint)


print("model loaded")
```

```
start bz prediction
model loaded
```

2.3 Perform Super -Resolution

Each input magnetogram is passed to the DeepHMI model for prediction. The output is a super-resolved magnetogram saved in the FITS format.

Here's the prediction and saving part:

In [4]:
```
for i in range(x_hmi_test.shape[0]):
    print("process # " + str(i+1) + " image")
    output_name = 'bz'+name_list[i][-19:]
    model.eval()
    x_test = torch.tensor(x_hmi_test[i:i+1, :1, :, :]).to(device)
    y_pred = np.random.normal(0, 1, x_hmi_test[i:i+1, :1, :, :].shape).astype(np.float32)
    y_pred = torch.tensor(y_pred, dtype=torch.float32).to(device)

    with torch.no_grad():
```

```
        for _ in range(beta.shape[0]):
            re_beta = np.array([np.flip(beta)[_]]).astype(np.float32)
            re_beta = torch.tensor(re_beta).to(device)
            y = model(y_pred, x_test, re_beta)
            re_beta = np.array([np.flip(beta)[_]]).astype(np.float32)
            re_beta = torch.tensor(re_beta).to(device)
            y_pred = (y_pred - y * torch.sqrt(re_beta)[:, None, None, None]) * (1 / torch.sqrt(1 - re_beta))[:, None, None, None]

    y_pred = y_pred.cpu().detach().numpy() * 1500

    total_fits_file = os.path.join(output_path, output_name)
    total_fits = fits.PrimaryHDU(y_pred[0, 0, :, :])
    total_fits.writeto(total_fits_file, overwrite=True)
```

```
process # 1 image
```

For transverse magnetogram, the process is similar to above:

In [5]:
```
print("start bt prediction")
try:
    checkpoint = torch.load(model_path + "model_bt")
    model.load_state_dict(checkpoint)
except:
    checkpoint = torch.load(model_path + "model_bt", map_location=torch.device('cpu'))
    model.load_state_dict(checkpoint)

print("model loaded")
```

```
start bt prediction
model loaded
```

In [6]:
```
for i in range(x_hmi_test.shape[0]):
    print("process # " + str(i+1) + " image")
    output_name = 'bt'+name_list[i][-19:]
    model.eval()
    x_test = torch.tensor(x_hmi_test[i:i+1, 1:2, :, :]).to(device)
    y_pred = np.random.normal(0, 1, x_hmi_test[i:i+1, 1:2, :, :].shape).astype(np.float32)
    y_pred = torch.tensor(y_pred, dtype=torch.float32).to(device)
    with torch.no_grad():
        for _ in range(beta.shape[0]):
            re_beta = np.array([np.flip(beta)[_]]).astype(np.float32)
            re_beta = torch.tensor(re_beta).to(device)
            y = model(y_pred, x_test, re_beta)
            re_beta = np.array([np.flip(beta)[_]]).astype(np.float32)
            re_beta = torch.tensor(re_beta).to(device)
            y_pred = (y_pred - y * torch.sqrt(re_beta)[:, None, None, None]) * (1 / torch.sqrt(1 - re_beta))[:, None, None, None]

    y_pred = (y_pred.cpu().detach().numpy() + 1) * 750

    total_fits_file = os.path.join(output_path, output_name)
    total_fits = fits.PrimaryHDU(y_pred[0, 0, :, :])
    total_fits.writeto(total_fits_file, overwrite=True)
```

```
process # 1 image
```

3. Conclusion

We present a deep learning model, DeepHMI, for enhancing AR LOS and transverse magnetograms of SOHO/MDI. This method can be applied to existing HMI data to produce high-resolution magnetic field maps.